TECHZEPHYR 4.0

SKETCH2SHAPE AI

From a 2D Sketch to a 3D Asset

24-Hour Live Hackathon | Computer Vision & Generative AI

1. Problem Background

Turning a creative idea into a detailed three-dimensional (3D) digital asset can be a challenging, time-consuming, and technically demanding process. Traditional 3D modelling often requires specialized skills in geometry creation, mesh editing, texturing, lighting, and rendering, along with familiarity with professional design software. This creates a barrier for individuals who have a concept in mind but lack the expertise or time required to convert it into a 3D representation.

A simple hand-drawn sketch is often the first step in expressing an idea. It may represent a product, household object, piece of furniture, architectural element, vehicle, toy, or an imaginative design. However, a two-dimensional sketch provides only limited information about the object's depth, volume, proportions, surface characteristics, and unseen geometry. Converting this incomplete visual representation into a meaningful 3D model requires interpretation, reconstruction, and creative decision-making.

The challenge becomes more complex when the sketch contains rough outlines, incomplete shapes, overlapping strokes, or limited visual details. A conventional image-processing system may identify edges and contours, but it cannot independently infer all the information needed to construct a complete 3D object. Similarly, a generative model may produce visually appealing results without accurately preserving the structure or intent of the original sketch.

1. The challenge

The challenge is to bridge this gap between a rough visual concept and a usable 3D digital asset by combining computer vision, generative AI, and 3D reconstruction techniques into a single, coherent application. Participants are expected to develop a functional application that accepts a single-view 2D sketch and transforms it into a visually compelling, textured 3D model. Users must be able to either draw their sketches directly within the application or upload existing sketch images. An optional text prompt may provide additional information about the intended object, its materials, colors, appearance, or artistic style.

The application should interpret the visual information in the sketch, generate or prepare suitable visual representations where required, and reconstruct a corresponding 3D asset. The resulting model should retain the primary structure and recognizable characteristics of the original concept while incorporating plausible geometry, surface details, and textures.

The solution should provide a practical, accessible way to explore ideas and create initial 3D assets without requiring users to perform the entire modelling process manually. Potential applications include:

- **Product design and prototyping**
- **Architecture and interior design**
- **Animation, VFX, and gaming**
- **3D printing**
- **Education and creative exploration**

2. Expected Solution from Participants

Participants are expected to develop a functional application that integrates the necessary components for converting a 2D sketch into an exportable 3D asset. The implementation may use different model architectures, libraries, and technical approaches, provided that the final system demonstrates a working
and reasonably efficient end-to-end pipeline.

A. Sketch Input and User Interface

The application must provide a simple and accessible interface through which users can supply the visual concept they want to convert into 3D.

- **In-app sketching:** Provide a drawing canvas that allows users to create a rough sketch using basic drawing tools, such as a brush, eraser, and canvas reset.
- **Image upload:** Allow users to upload an existing sketch image in a commonly supported image format.
- Input preparation: Support appropriate preprocessing, such as resizing, background handling, contrast adjustment, or noise reduction, where required.
- **Optional text prompt:** Allow users to describe additional characteristics of the intended object, including its type, material, color, style, or other visual attributes. This feature is optional and must not be a prerequisite for generating a 3D model.
- The interface should make it clear how a user can provide a sketch and obtain the resulting 3D asset. A conversational chatbot or chat-based interface is not mandatory.

B. Sketch Understanding and Visual Generation

The system should use suitable computer-vision and generative-AI techniques to interpret the supplied sketch and prepare visual information for 3D reconstruction.

The solution may incorporate techniques for:

- Identifying the major contours, shapes, and structural features of the sketch.
- Preserving the key visual characteristics of the original input during generation.
- Using a text prompt, when available, to guide the appearance, materials, colors, or style of the intended object.
- Generating or refining a suitable visual representation for the subsequent reconstruction stage.
- Preparing the image through operations such as background removal, segmentation, or other relevant image-processing techniques.

- The generated representation should remain meaningfully related to the input sketch rather than producing an unrelated object solely based on a text prompt.

C. 3D Model Reconstruction and Texturing

The application must incorporate a suitable 3D reconstruction component capable of converting the prepared visual representation into a three-dimensional digital asset.

The reconstructed model should aim to provide:

Recognizable geometry: The overall shape and primary structural features should correspond to the input sketch.

Three-dimensional form: The output should contain meaningful depth and volume rather than being only a flat image or plane.

Visual quality: The model should have a coherent appearance, with suitable proportions and reasonably smooth or well-formed surfaces where appropriate.

Texture and appearance: Where supported by the chosen approach, the asset should include suitable colors, materials, or textures that improve its visual presentation.

Usable output: The resulting asset should be accessible for inspection and further refinement in compatible 3D software.

Since a single-view sketch does not fully specify the object's unseen surfaces, some geometric interpretation is unavoidable. Participants should aim for plausible and visually consistent reconstruction rather than assuming that the hidden geometry can always be recovered exactly.

D. 3D Preview and Export

- The generated asset must be exportable in at least one commonly supported 3D file format, such as .glb or .obj.
- The application should ideally provide a way to preview the generated model, inspect it from different angles, and download or save the output. An embedded interactive 3D viewer is encouraged but not mandatory if the exported asset can be opened and inspected using compatible software.
- The output should represent actual 3D geometry. Exporting only a rendered image, a screenshot, or a flat textured plane will not satisfy the intended objective.
- Participants should also consider basic mesh validation, including checking whether the exported file can be opened and whether the geometry and textures are preserved as intended.

Suggested Solution Pipeline

The following pipeline is a suggested direction, not a prescribed implementation. Teams may propose alternative models, libraries, or architectures as long as the submitted system meets the expected outcome.

Pipeline component	Direction
Input and sketch conditioning	Accept an in-app drawing or uploaded image; optionally incorporate a text prompt and derive structural cues such as edges or scribbles.
Sketch-guided image generation	Generate a refined product-style image guided by the sketch and optional prompt. One possible approach is Stable Diffusion with ControlNet.
Image preparation	Prepare the generated image for reconstruction, including foreground isolation or background removal where required.
Image-to-3D reconstruction	Use a pretrained image-to-3D model, such as InstantMesh, Unique3D, or an equivalent approach, to estimate geometry and surface appearance.
Mesh validation and export	Validate that the output contains usable geometry and export a 3D asset in .glb or .obj format, preserving materials or textures where supported.

Possible supporting technologies include PyTorch, Hugging Face Diffusers, ControlNet, OpenCV, rembg, trimesh, and a lightweight interface framework such as Gradio or Streamlit. These are examples only; use of any particular model or framework is not mandatory. Teams must disclose pretrained models, external APIs, and third-party components used.

Scope and Constraints

- The total development period is 24 hours; teams should prioritize a working end-to-end implementation over optional features.
- Pretrained models and open-source libraries may be used, subject to event rules and relevant licenses.
- Either or both the input options—drawing a sketch in the application and uploading a sketch image— is allowed.
- At least one valid output format, .glb or .obj, must be delivered.
- The solution must include clear setup instructions and a reproducible way to run the pipeline.
- Manufacturing-ready 3D-printing geometry, engineering-grade architectural models, and production-ready animation topology are outside the mandatory scope.

Judging Criteria

- Judges may provide a fresh sketch or select a test case to verify the pipeline is not relying only on pre-generated outputs.
- The final 3D file will be checked for successful opening, non-empty geometry, and usable export.
- Output quality will be assessed against the input sketch and prompt, considering form, visual coherence, and finish.
- Reproducibility will be assessed by following the README and setup instructions in the available evaluation environment.
- Teams should report hardware assumptions and limitations.

Submission Requirements

- **Working prototype:** Demonstrate the end-to-end application with either both sketch drawing and image upload as a web application, locally hosted application, desktop application, or command-line tool.
- **Source code:** Submit the code needed to reproduce the pipeline, including preprocessing, model integration, and export logic.
- **README and documentation:** Explain the objective, architecture, dependencies, installation, execution, model choices, expected inputs/outputs, and also report the time required to generate a 3D model, along with the hardware and computational resources used ***(Report about your limitations for the same in the documentations itself if have any issue in generating the model)***
- **Output artefact:** Submit at least one generated .glb or .obj file and example input/output results.
- **Reproducibility details:** Document environment requirements, model-weight access, external services, and configuration needed to reproduce a run.
- **Final demonstration:** Show the pipeline operating end-to-end and explain key implementation decisions.

Participants may implement the solution as a web application, locally hosted application, desktop application, or command-line tool. Developing or deploying a public website is not mandatory. However, the solution must support the required sketch inputs, demonstrate 3D model generation, and export the resulting asset in a supported format.

Evaluation Rubric — 100 Points

Projects will be assessed on engineering execution, documentation, output quality, reproducibility, and end-to-end pipeline performance. A polished interface alone will not substitute for a functioning and reproducible system.

Criterion	Points	Judging focus
Technical implementation	**25**	Quality of model integration, code structure, preprocessing, error handling, and technical decisions.
3D output quality and design	**25**	Visual appeal, recognisability, fidelity to sketch and prompt, plausible geometry, and texture/material quality.
End-to-end pipeline validity and efficiency	**20**	Successful operation from drawing/upload through reconstruction and export; reliability, runtime, and resource use.
Code reproducibility	**15**	Judges can set up and run the project using submitted instructions and disclosed dependencies, without undocumented manual steps.
Documentation and README quality	**15**	Clear overview, architecture, setup, execution, model/data attribution, sample results, limitations, and troubleshooting.

Suggested References

- Hugging Face Diffusers — ControlNet: https://huggingface.co/docs/diffusers/main/en/using-diffusers/controlnet
- InstantMesh repository: https://github.com/TencentARC/InstantMesh
- Unique3D repository: https://github.com/AiuniAI/Unique3D
- trimesh documentation: https://trimesh.org/

TECHZEPHYR 4.0 • IIT BHUBANESWAR